// THE SERVICE AND THE SCREEN, PART ONE

An API with FastAPI

Why the browser cannot call our Python directly, and the small app that fixes it



// 23 FASTAPI

Why an API



the API is the front desk: it takes a request, calls the right function, and returns the answer

a browser cannot call ask_question() directly. It can send an HTTP message.

// 23 FASTAPI

One request, one reply

QUESTION **What does the server send back?**

GET /hello | Path and query | A valid order | A field is missing | A wrong value | Our own refusal

THE REQUEST: WHAT THE BROWSER SENDS

```
GET /hello HTTP/1.1
Host: localhost:8001
```

THE REPLY: WHAT THE SERVER SENDS BACK

200 OK

200 OK

```
{"message":"hello"}
```

```
Uvicorn logged: INFO: 127.0.0.1:58097 - "GET /hello HTTP/1.1" 200 OK
```

Real requests and replies of the hello app on the next slides, recorded on 10 October 2026. The client port in the log lines differs on every run.

// 23 FASTAPI

The technology behind the server

The server stack

SHOW ONLY Framework Server Validation Documentation Tools

8 shown

FastAPI Used in Meridian

The web framework: turns Python functions into routes, checks the input, and writes the documentation.

GOOD FOR APIs written with typed Python.

Framework

Uvicorn Used in Meridian

The server process: it listens on a port, speaks HTTP, and hands each request to the app.

GOOD FOR Running FastAPI locally and in Docker.

Server

Starlette Used in Meridian, under FastAPI

The small toolkit under FastAPI: routing, middleware, static files and the test client.

GOOD FOR Everything FastAPI does not add itself.

Framework

Pydantic Used in Meridian

Checks data against a class with typed fields, converts it, and explains what is wrong.

GOOD FOR Request and reply shapes.

Validation

Python type hints Used in every file

The annotations such as name: str and quantity: int that FastAPI and Pydantic read.

GOOD FOR Telling the framework what you expect.

Validation

OpenAPI Generated for us at /openapi.json

A standard way to describe an API as one JSON file: routes, inputs, outputs.

GOOD FOR Documentation and generated clients.

Documentation

Swagger UI Served for us at /docs

The web page that reads the OpenAPI file and lets you try every route in the browser.

GOOD FOR Testing an API without a frontend.

Documentation

curl Used in these chapters

A command-line tool that sends one HTTP request and prints the reply.

GOOD FOR Quick checks from a terminal.

Tools

Who does what when a request arrives



five layers; you write only the last one

the reply travels back through the same layers

// 23 FASTAPI

The app, part 1: the shapes

backend/try_hello.py (lines 13 to 27 of 47)

```python
from fastapi import FastAPI, HTTPException  1
from pydantic import BaseModel, Field

app = FastAPI(title="FastAPI hello")  2


class OrderRequest(BaseModel):  3
    """The shape of the data we expect. FastAPI checks every request against it."""
    item: str
    quantity: int = Field(gt=0, description="Must be greater than zero")  4


class OrderResponse(BaseModel):  5
    message: str
    total_items: int
```

1 The imports
FastAPI is the application. HTTPException turns an error into an HTTP reply. BaseModel and Field come from Pydantic.

2 The application
One object is the whole service. Uvicorn looks for this name: the :app in the start command.

3 A request shape
A class with typed fields. FastAPI checks the body of every request against it.

4 A rule as data
An integer greater than zero. The rule is declared, not written as an if statement.

5 A reply shape
The reply is checked too, through response_model on the route below.

The app, part 2: the routes

backend/try_hello.py (lines 30 to 47 of 47)

```python
@app.get("/hello")  1
def hello():
    return {"message": "hello"}


@app.get("/hello/{name}")  2
def hello_name(name: str, shout: bool = False):  2
    # {name} comes from the URL path; shout comes from ?shout=true in the query string
    text = f"hello {name}"
    return {"message": text.upper() if shout else text}


@app.post("/orders", response_model=OrderResponse)  3
def create_order(order: OrderRequest):  3
    if order.item.strip().lower() == "forbidden":
        # HTTPException turns into a proper HTTP error response
        raise HTTPException(status_code=400, detail="That item cannot be ordered")  4
    return OrderResponse(message=f"Ordered {order.quantity} x {order.item}",
total_items=order.quantity)  5
```

1 A route
Answer GET /hello with this function. The return value becomes JSON.

2 Path and query
{name} becomes the argument; shout has a default, so ?shout=true is optional.

3 A POST route
The body has a Pydantic type, so FastAPI reads the JSON into it or refuses. response_model checks the reply.

4 Our own refusal
HTTPException becomes an error reply with our status code.

5 The reply
A normal object; FastAPI makes it JSON.

// 23 FASTAPI

Run it: copy and start

project folder, terminal 1 Copy

```
$ cp ../meridian-reference/docs/examples/01_fastapi_hello.py backend/try_hello.py
$ wc -l backend/try_hello.py
47 backend/try_hello.py

# start the server; it stays busy
$ uv run uvicorn try_hello:app --app-dir backend --port 8001
INFO:     Started server process [3711]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
```

> Port 8001 for this example, so port 8080 stays free for the real server.

Copy

The example comes from the repository's docs/examples folder.

Uvicorn

try_hello:app means: the file try_hello.py, the object app.

It stays busy

That is the server waiting. Open a second terminal.

// 23 FASTAPI

Call it: four requests

project folder, terminal 2 Copy

```
$ curl http://localhost:8001/hello
{"message":"hello"}
$ curl "http://localhost:8001/hello/Asha?shout=true"
{"message":"HELLO ASHA"}
$ curl -X POST http://localhost:8001/orders -H "Content-Type: application/json" -d '{"item": "servo motor",
"quantity": 2}'
{"message":"Ordered 2 x servo motor","total_items":2}
$ curl -X POST http://localhost:8001/orders -H "Content-Type: application/json" -d '{"item": "servo motor"}'
{"detail":[{"type":"missing","loc":["body","quantity"],"msg":"Field required","input":{"item":"servo
motor"}}]}
```

> Windows PowerShell: use curl.exe and escape the quotes (see the reference).

Four of the six

The widget showed all six; the reference lists them.

Or the browser

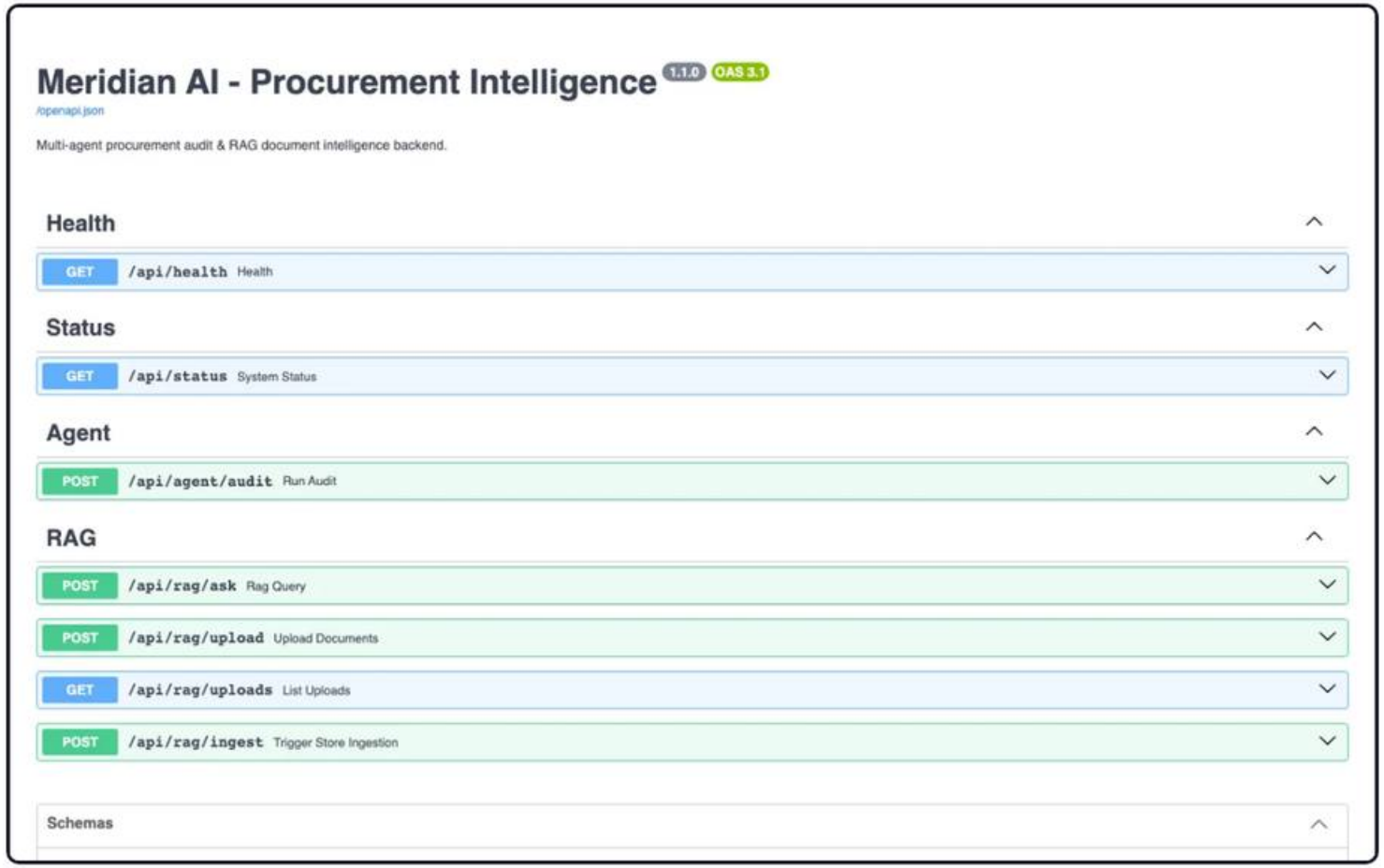
Open http://localhost:8001/docs and use Try it out.

Stop it

Ctrl+C in the first terminal.

// 23 FASTAPI

Free documentation, at /docs



The /docs page of our real server (built in the next chapter)

Made from your code

FastAPI reads the routes and shapes and writes an OpenAPI file at /openapi.json.

Try it out

Every route has a button that sends a real request.

Useful to others

A teammate or a frontend developer can learn the API from this page.

// SUMMARY

FastAPI in five lines

1 **An API is a front desk.**
It turns an HTTP request into a function call.

2 **A route is a decorated function.**
The path, the method and the types come from the code.

3 **Pydantic checks the shape.**
A bad request gets a 422 that names the field; our code never runs.

4 **Uvicorn runs it.**
try_hello:app is the file and the object.

5 **/docs is free.**
Generated from the same code, so it cannot drift.

Request shapes and routes

Two files: what a request must look like, and the thin functions that answer it



Refuse a bad request politely

A good request | A bad request



1. A JSON body arrives

The screen sends the question as JSON. The route's parameter says payload: QueryRequest.

```
{"query": "What are the payment terms?"}
```

Back | Next step | Play

A good request: step 1 of 5

Put the files in place

project folder

Copy

```
$ cp ../meridian-reference/backend/api/schemas.py backend/api/schemas.py
$ cp ../meridian-reference/backend/api/endpoints.py backend/api/endpoints.py
$ wc -l backend/api/schemas.py backend/api/endpoints.py
  29 backend/api/schemas.py
 203 backend/api/endpoints.py
 232 total
```

> Windows PowerShell: use Copy-Item instead of cp.

Two files

schemas.py is 29 lines; endpoints.py is 203.

Do not run yet

endpoints.py needs main.py, which comes in the next chapter.

Or type them

Follow the slides and type the code yourself.

schemas.py: four shapes

backend/api/schemas.py (lines 10 to 30 of 30)

```python
class AuditRequest(BaseModel):    # 1
    request_text: str

class AuditResponse(BaseModel):    # 2
    risk_result: str
    tax_result: str
    control_result: str
    cfo_memo: str


# ============================================================
# RAG  Schemas
# ============================================================


class QueryRequest(BaseModel):    # 3
    query: str
    retriever_type: str = "similarity"    # 4

class QueryResponse(BaseModel):    # 5
    answer: str
```

1 The audit request
One field: the purchase request, as text.

2 The audit reply
The three specialist reports and the CFO memo, all text.

3 The question
query is required: it has no default.

4 An optional choice
retriever_type is similarity unless the caller asks for multi-query or compression (Module 4).

5 The answer
One field of text.

Try it: a shape without a server

backend/try_schemas.py (create this file) Copy

```python
from pydantic import ValidationError

from api.schemas import QueryRequest

ok = QueryRequest(query="What are the payment terms?")  # 1
print("valid  :", ok)
print("as dict:", ok.model_dump())  # 2

try:
    QueryRequest(retriever_type="similarity")  # 3
except ValidationError as err:
    first = err.errors()[0]  # 4
    print("invalid:", first["type"], first["loc"], "—", first["msg"])
```

1. **Build one**
   Give it a question; the optional field is filled in.
2. **Back to a dict**
   model_dump gives plain data: this is what becomes JSON.
3. **Break it**
   Leave out the question and Pydantic raises ValidationError.
4. **Read the error**
   A type, a location and a message: the same facts FastAPI sends in the 422 reply.

// 24 REQUEST SHAPES AND ROUTES

Run it: valid and invalid

project folder Copy

```
$ uv run python backend/try_schemas.py
valid   : query='What are the payment terms?' retriever_type='similarity'
as dict: {'query': 'What are the payment terms?', 'retriever_type': 'similarity'}
invalid: missing ('query',) - Field required
```

> No server, no key, no internet.

Valid
The default retriever_type appears by itself.

Invalid
The error names the missing field.

No network
Nothing here talked to Gemini or to a server.

// 24 REQUEST SHAPES AND ROUTES

Seven routes, one job each

METHOD AND PATH	FUNCTION	WHAT IT CALLS	ASKS GOOGLE?
GET /api/health	health	nothing	No
GET /api/status	system_status	settings only	No: safe without a key
POST /api/agent/audit	run_audit	ProcurementSupervisor.run_audit	Yes: about a minute
POST /api/rag/ask	rag_query	ask_question	Yes: a few seconds
POST /api/rag/upload	upload_documents	ingest_pdf and document_store.save	Yes: embeddings
GET /api/rag/uploads	list_uploads	an in-memory list	No
POST /api/rag/ingest	trigger_store_ingestion	ingest_stored_pdfs	Yes: embeddings



a hidden eighth route, ingest-gcs, is an old name for the last one

endpoints.py: the set-up

backend/api/endpoints.py (lines 34 to 47)

```python
health_router = APIRouter(prefix="/api", tags=["Health"])  1
status_router = APIRouter(prefix="/api", tags=["Status"])
agent_router = APIRouter(prefix="/api/agent", tags=["Agent"])
rag_router = APIRouter(prefix="/api/rag", tags=["RAG"])
# ...
# In-memory upload tracker (reset on restart)
_upload_history: list[dict] = []  2
_server_start_time = datetime.now(timezone.utc)  3
# ...
@lru_cache(maxsize=1)  4
def get_supervisor() -> ProcurementSupervisor:
    """Build the agents on the first audit request, not when the server starts."""
    return ProcurementSupervisor()  5
```

1 **Four routers**
A prefix and a tag each. The tags become the groups on the /docs page.

2 **Upload history**
A list in memory: it resets when the server restarts.

3 **Start time**
Used to report the uptime.

4 **lru_cache**
Build the agents once, and only when first needed.

5 **Why lazy**
The agents need the key. The server starts, and /api/status works, without it.

Health and status: no AI calls

backend/api/endpoints.py (lines 54 to 100, shortened)

```python
@health_router.get("/health")  1
def health():
    return {"status": "ok", "mode": settings.app_mode}
# ...
@status_router.get("/status")  2
def system_status():
    """Return system configuration and health information (makes no calls to Google)."""
    uptime_seconds = int((datetime.now(timezone.utc) - _server_start_time).total_seconds())  3
    hours, remainder = divmod(uptime_seconds, 3600)
    minutes, seconds = divmod(remainder, 60)
    cloud = settings.is_cloud

    return {
# ...
"vector_search": {
    "backend": "Vertex AI Vector Search" if cloud else "Chroma (local disk)",  4
    "index_id": (settings.vector_search_index_id or "NA") if cloud else settings.chroma_dir,
# ...
"ingestion": {
    "uploads_this_session": len(_upload_history),  5
},
```

1. **Health**
   Returns at once: is the server up, and in which mode.

2. **Status**
   A summary of the configuration. It makes no calls to Google.

3. **Uptime**
   Computed from the start time, shown as hours, minutes and seconds.

4. **Two modes**
   Every value has a local and a cloud branch. Locally, the cloud values read 'not used'.

5. **Counters**
   The System Status tab shows these, and refreshes every 5 seconds.

Audit and ask: thin routes

backend/api/endpoints.py (lines 108 to 128, shortened)

```python
@agent_router.post("/audit", response_model=AuditResponse)   1
def run_audit(payload: AuditRequest):
    """Run the full multi-agent procurement audit and return all phase results."""
    try:
        return AuditResponse(**get_supervisor().run_audit(payload.request_text))   2
    except MeridianError:   3
        raise  # handled in api/main.py
    except Exception as e:
        log.exception("Audit failed")   4
        raise HTTPException(status_code=500, detail=str(e))   5
# the ask route repeats the pattern:
@rag_router.post("/ask", response_model=QueryResponse)
def rag_query(payload: QueryRequest):
    """Ask a question; the answer comes only from the indexed documents."""
    try:
        return QueryResponse(answer=ask_question(payload.query, payload.retriever_type))
```

1 A route with a reply shape
post, and response_model: the reply must match AuditResponse.

2 One line of work
Call the real code and wrap the result. All the logic lives in agent/ and rag/.

3 Our errors pass through
A ConfigurationError becomes a 503 in main.py (next chapter).

4 A traceback in the log
log.exception writes the full stack trace, so a crash can be diagnosed later.

5 Anything else is a 500
The caller gets a message; the details stay in the log.

Upload, part 1: receive and sort

backend/api/endpoints.py (lines 136 to 155)

```python
@rag_router.post("/upload")
async def upload_documents(files: list[UploadFile] = File(...)):  1
    """Accept one or more PDFs, keep a copy in the document store, then chunk, embed and index
them."""
    if not files:
        raise HTTPException(status_code=400, detail="No files provided.")

    results = []
    tmp_dir = tempfile.mkdtemp()  2
    try:
        for upload in files:
            filename = os.path.basename(upload.filename or "")  3
            if not filename.lower().endswith(".pdf"):  4
                results.append({"filename": filename, "status": "skipped",
                                "reason": "Only PDF files are supported."})
                continue

            # Save to a temp file (the async UploadFile must be awaited to read it)
            local_path = os.path.join(tmp_dir, filename)
            with open(local_path, "wb") as f:
                f.write(await upload.read())  5
```

1 Async, many files
async because reading an upload is awaited; the form may hold several files.

2 A temp folder
Everything for this request goes into one folder that is deleted at the end.

3 Only the file name
basename drops any folder part, so a name like ../../x cannot escape the folder.

4 Skip, do not fail
A file that is not a PDF is reported as skipped with a reason.

5 Save the bytes
await reads the upload; the file is written to the temp folder.

Upload, part 2: store, index, clean up

backend/api/endpoints.py (lines 158 to 180, shortened)

```python
try:
    await run_in_threadpool(document_store.save, local_path, filename)  # 1
except Exception as store_err:
    log.warning("Could not store original file", filename=filename, error=str(store_err))
# ...
summary = await run_in_threadpool(ingest_pdf, local_path, f"upload://{filename}")  # 2
# ...
if summary["chunks"] == 0:  # 3
    log.warning("No chunks extracted, skipping indexing", filename=filename)
# ...
results.append({"filename": filename, "status": "ingested", **summary})  # 4
# ...
finally:
    shutil.rmtree(tmp_dir, ignore_errors=True)  # 5
```

1. **Keep the original**
   Saved to the document store. If that fails, indexing still goes ahead; a warning is logged.

2. **A worker thread**
   ingest_pdf is slow and blocking. In a thread, the server keeps answering other requests.

3. **No text, no index**
   A scanned PDF with no text is skipped with a reason.

4. **One result per file**
   ingested, with the page and chunk counts.

5. **Always clean up**
   finally runs on success and on error: the temp folder is removed.

// SUMMARY

Shapes and routes in five lines

1. **A shape is a contract.**
   Pydantic checks every field before our code runs.

2. **A bad request gets a 422 that names the field.**
   No model call, no cost.

3. **A route is thin.**
   Validate, call the real code, return.

4. **Only some routes ask Google.**
   Health, status and the upload list never do.

5. **Slow work goes to a thread.**
   The server keeps answering while a PDF is embedded.

The server: assemble it, run it

One file plugs the routes together and turns errors into friendly replies. Then we start it and try to break it



Put main.py in place

project folder

Copy

```
$ cp ../meridian-reference/backend/api/main.py backend/api/main.py
$ wc -l backend/api/main.py
91 backend/api/main.py
```

> Windows prints just 91.

91 lines

The shortest of the three API files.

It plugs in

The four routers, the error handlers and the frontend.

Or type it

Follow the slides.

// 25 THE SERVER

main.py: start-up, the app, CORS

backend/api/main.py (lines 30 to 51, shortened)

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    log.info("Meridian AI starting", mode=settings.app_mode, llm=settings.llm_model_name)  1
    yield  # the app runs here  2
    # Shutdown (cloud mode only): flush logs to GCS, because Cloud Run's disk disappears
    _LOGGER_INSTANCE.flush_to_gcs()  3
# ...
app = FastAPI(  4
    title=settings.app_name,
    version=settings.app_version,
    description="Multi-agent procurement audit & RAG document intelligence backend.",
    lifespan=lifespan,
)
# ...
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],           # Tighten in production  5
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

1 Start-up
The first log line: the mode and the model.

2 The app runs
Everything before yield is start-up, everything after is shutdown.

3 Shutdown
Cloud mode only: upload the log file, because a container's disk disappears.

4 The application
Title and version come from settings and show at the top of /docs.

5 CORS
'*' lets any site call this API: a demo setting. Tighten it before deploying (Part 2).

// 25 THE SERVER

CORS: the browser asks first

project folder Copy

```
$ curl -i -H "Origin: http://localhost:3000" http://localhost:8080/api/health
HTTP/1.1 200 OK
server: uvicorn
content-type: application/json
access-control-allow-origin: http://localhost:3000
access-control-allow-credentials: true
vary: Origin

{"status":"ok","mode":"local"}
```

> A browser on another origin sends the Origin header and looks for access-control-allow-origin in the reply.

An origin

Scheme, host and port. The screen is localhost:3000; the API is localhost:8080: two origins.

The browser asks

It blocks the reply unless the server says that origin is allowed.

Not a lock

CORS protects browsers only. curl ignores it. Never rely on it for security.

// 25 THE SERVER

Every failure has a reply and a log line

WHAT THE USER SEES

- Ask a question with an empty key
- Upload a corrupt PDF
- Send a request without the query
- Call a route that does not exist
- Upload a file that is not a PDF
- Ask with an unknown retriever type
- Something unexpected breaks (a bug)

DIAGNOSIS

LIKELY CAUSE

Gemini needs a key; the settings found none (ConfigurationError).

WHAT TO TRY

Put GOOGLE_API_KEY in .env.

IN MERIDIAN

503 {"detail": "GOOGLE_API_KEY is empty. Add it to .env (...)"}. Handler configuration_error in main.py; log: "Configuration error", severity ERROR.

main.py: errors become replies

backend/api/main.py (lines 53 to 69, shortened)

```python
@app.exception_handler(ConfigurationError)  # 1
async def configuration_error(_: Request, exc: ConfigurationError):
    """A missing setting is not a crash: tell the caller exactly what to set."""
    log.error("Configuration error", error=str(exc))
    return JSONResponse(status_code=503, content={"detail": str(exc)})  # 2
# ...
@app.exception_handler(IngestionError)  # 3
async def ingestion_error(_: Request, exc: IngestionError):
    log.error("Ingestion error", error=str(exc))
    return JSONResponse(status_code=422, content={"detail": str(exc)})  # 4
# ...
app.include_router(health_router)  # 5
app.include_router(status_router)
app.include_router(agent_router)
app.include_router(rag_router)
```

1. **One handler per error**
   FastAPI calls this function whenever a ConfigurationError escapes a route.

2. **503**
   Service unavailable: the server is fine, a setting is missing. The message says how to fix it.

3. **A second error type**
   A file we cannot read.

4. **422**
   The caller sent something we cannot use. Each handler first writes a log line at level ERROR.

5. **The routers**
   The four routers from endpoints.py are plugged in here.

main.py: serve the screen

backend/api/main.py (lines 76 to 91)

```python
frontend_dist = os.path.realpath(os.path.join(os.path.dirname(__file__), "../../frontend/dist"))

if os.path.exists(frontend_dist):  # 1
    app.mount("/assets", StaticFiles(directory=os.path.join(frontend_dist, "assets")),
name="assets")  # 2

    # Catch-all: serve a real file if it exists, otherwise index.html (React handles the route)
    @app.get("/{catchall:path}", include_in_schema=False)  # 3
    def serve_frontend(catchall: str):
        if catchall.startswith("api/"):  # 4
            raise HTTPException(status_code=404, detail="Not found")
        file_path = os.path.realpath(os.path.join(frontend_dist, catchall))
        if file_path.startswith(frontend_dist + os.sep) and os.path.isfile(file_path):  # 5
            return FileResponse(file_path)
        return FileResponse(os.path.join(frontend_dist, "index.html"))
# ...
else:
    log.warning("Frontend dist not found — frontend will not be served.", path=frontend_dist)
```

1. **Only if it was built**
   No frontend/dist folder: log a warning and serve nothing (the warning we saw).

2. **/assets**
   The built JavaScript and CSS files.

3. **A catch-all**
   Any other path returns a real file, or index.html: React decides what to show.

4. **API paths stay API**
   A path starting api/ that matched no route is a real 404 in JSON, never the web page.

5. **No escaping**
   realpath and startswith: a path such as ../../.env cannot leave the folder.

// 25 THE SERVER

Run it: start the server

project folder

Copy

```
$ uv run uvicorn api.main:app --app-dir backend --reload --port 8080
INFO:     Will watch for changes in these directories: ['/Users/you/meridian-ai']
INFO:     Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
INFO:     Started reloader process [3957] using WatchFiles
{"path": "/Users/you/meridian-ai/frontend/dist", "timestamp": "2026-10-10T19:42:23.633034Z",
"level": "warning", "severity": "WARNING", "event": "Frontend dist not found - frontend will
not be served."}
INFO:     Started server process [3959]
INFO:     Waiting for application startup.
{"mode": "local", "llm": "gemini-3.8-flash", "timestamp": "2026-10-10T19:42:23.633769Z",
"level": "info", "severity": "INFO", "event": "Meridian AI starting"}
INFO:     Application startup complete.
```

> Two JSON lines are ours, the rest is Uvicorn's. More on that in the logging chapter.

api.main:app

The file backend/api/main.py, the object app.

--app-dir backend

Where to look for the package api.

--reload

Restart on every saved Python file. For development only.

// 25 THE SERVER

Call it: health, status, upload

project folder

Copy

```
$ curl http://localhost:8080/api/health
{"status":"ok","mode":"local"}
$ curl http://localhost:8080/api/status
{"mode":"local","backend":{"status":"healthy","uptime":"00h 00m
21s","python_version":"3.12.11","platform":"Darwin"},"gcp":{...},"storage":
{...},"vector_search":{"backend":"Chroma (local disk)",...},"models":{"embedding":"gemini-
embedding-001","llm":"gemini-3.8-flash",...},"ingestion":{"uploads_this_session":0}}
$ curl -X POST http://localhost:8080/api/rag/upload -F
"files=@sample_docs/aldermoor_procurement_policy.pdf"
{"results":
[{"filename":"aldermoor_procurement_policy.pdf","status":"ingested","pages":1,"chunks":1}]}
$ curl -X POST http://localhost:8080/api/rag/upload -F "files=@pyproject.toml"
{"results":[{"filename":"pyproject.toml","status":"skipped","reason":"Only PDF files are
supported."}]}
```

> The status reply is shortened on the slide (the real one lists the folders of your machine).

Health

Instant, no key: the server is up.

Status

A summary of the setup; no Google call.

Upload

A PDF is embedded (about 5 s). A .toml is skipped with a reason.

// 25 THE SERVER

Ask a question: RAG over HTTP

project folder Copy

```
$ curl -X POST http://localhost:8080/api/rag/ask -H "Content-Type:
application/json" -d '{"query": "What payment terms do servo motor suppliers
get?"}'
{"answer":"Servo motor suppliers with a credit score of 75 or higher (low risk)
receive standard **Net-45** payment terms. ... "}
```

> Shortened on the slide; the real answer continues with the lower-score tiers.

The same pipeline
ask_question from Module 4, now behind a route.

A few seconds
One embedding call and one model call.

Shortened
The real answer goes on to the lower-score tiers.

// 25 THE SERVER

Break it: four failures

project folder Copy

```
$ curl -X POST http://localhost:8080/api/rag/ask -H "Content-Type: application/json" -d '{}'
{"detail":[{"type":"missing","loc":["body","query"],"msg":"Field required","input":{}}]}
$ curl http://localhost:8080/api/nothing
{"detail":"Not Found"}
$ echo "this is not a pdf" > fake.pdf
$ curl -X POST http://localhost:8080/api/rag/upload -F "files=@fake.pdf"
{"detail":"Could not read PDF 'upload://fake.pdf': Stream has ended unexpectedly"}
$ rm fake.pdf .meridian/uploads/fake.pdf
```

> Missing field: 422 from FastAPI before our code runs.

422 from FastAPI

The missing field is named. Our code never ran.

404

A route that does not exist.

422 from our handler

A corrupt PDF. The last command removes it: the original is stored first, so it would stay.

// 25 THE SERVER

Break it: no key

project folder Copy

```
$ GOOGLE_API_KEY= uv run uvicorn api.main:app --app-dir backend --port 8081
# in another terminal:
$ curl -X POST http://localhost:8081/api/rag/ask -H "Content-Type: application/json" -d
'{"query": "hello"}'
{"detail":"GOOGLE_API_KEY is empty. Add it to .env
(https://aistudio.google.com/apikey)."}
$ curl http://localhost:8081/api/health
{"status":"ok","mode":"local"}
```

> An environment variable beats the value in .env, so the empty value wins for this one process.

An empty key

The environment variable wins over .env for this one process.

503, not a crash

The reply says exactly what to set.

Health still answers

The server is up; only Gemini is unavailable.

// 25 THE SERVER

Run an audit through the API

project folder

Copy

```
$ curl -X POST http://localhost:8080/api/agent/audit -H "Content-Type: application/json" -d
'{"request_text": "Purchase request: Vendor Mitsubishi Electric (Japan). Item: 20 spare servo motors
for the Aldermoor filling line. Total cost: 38,000 EUR. Origin: JP. Destination: DE. FX rate quoted:
1 EUR = 176.5 JPY."}' -s -o .meridian/audit.json -w "status %{http_code} in %{time_total} s\n"
$ uv run python -c "import json; d = json.load(open('.meridian/audit.json')); print(list(d));
print(d['cfo_memo'][:200])"
status 200 in 63.170085 s
['risk_result', 'tax_result', 'control_result', 'cfo_memo']
═══════════════════════════════════════════════
ALDERMOOR INDUSTRIES — CFO AUDIT MEMORANDUM
Date: 24 May 2024
RE: Procurement Request Audit Summary
```

> Windows PowerShell: use curl.exe and escape the quotes (see the reference).

About a minute

Our runs: 63, 78 and 92 s. The server answers other requests meanwhile.

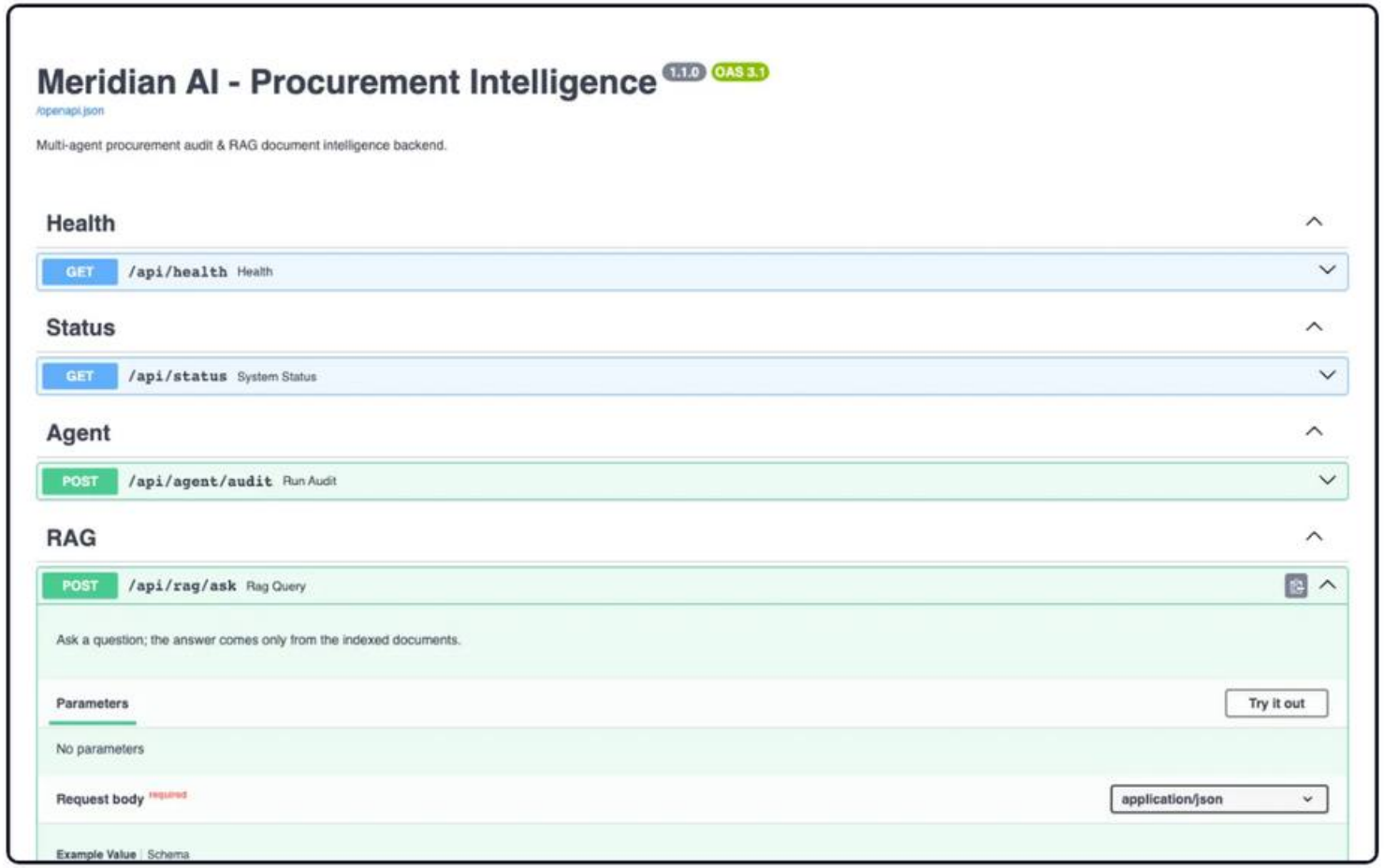
Four fields

The three specialist reports and the CFO memo: the AuditResponse shape.

Saved in .meridian

Git ignores it.

Or use /docs instead of curl



http://localhost:8080/docs: the ask route, opened

Try it out
Edit the example body and press Execute: a real request.

The shapes
The example body comes from QueryRequest.

Any system
The browser needs no curl, so this works the same on Windows.

// SUMMARY

The server in five lines

1 **main.py assembles the service.**
Routers, error handlers, CORS and the frontend.

2 **Errors become replies.**
503 for a missing setting, 422 for a bad input, 500 for a bug.

3 **CORS is a browser rule, not a lock.**
The star is a demo setting.

4 **One command starts it.**
uv run uvicorn api.main:app --app-dir backend --reload --port 8080

5 **We broke it on purpose.**
Every failure gave a clear reply. Next: what the log says about each.

// THE SERVICE AND THE SCREEN, PART FOUR

Reading the log

What the server says, where each kind of line goes, and how to find a problem afterwards



Why log at all



A record

At 2 a.m. the audit gave a strange answer. Without a record, you can only guess what happened.



Searchable

Each fact is its own field, not a sentence: you can search for a tool, a duration or a file name.



Levels

Info, warning, error: read the serious ones first.



The same everywhere

The same JSON lines locally and in the cloud, where tools can read them.

// 26 LOGGING

Three kinds of lines in one console

SHOW [Our lines (JSON)] [Uvicorn (start-up and access)] [Libraries (httpx, Gemini)] LEVEL [Info] [Warning] [Error] 22 of 22 lines

```
UVICORN INFO: Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
OURS {"path": "/Users/you/meridian-ai/frontend/dist", "timestamp": "2026-10-10T19:28:44.171189Z", "level": "warning", "severity": "WARNING", "event": "Frontend dist not found — frontend will not be served."}
OURS {"mode": "local", "llm": "gemini-3.8-flash", "timestamp": "2026-10-10T19:28:44.172480Z", "level": "info", "severity": "INFO", "event": "Meridian AI starting"}
UVICORN INFO: Application startup complete.
UVICORN INFO: 127.0.0.1:57701 - "GET /api/health HTTP/1.1" 200 OK
OURS {"filename": "aldermoor_procurement_policy.pdf", "stored_at": "/Users/you/meridian-ai/.meridian/uploads/aldermoor_procurement_policy.pdf", "timestamp": "2026-10-10T19:28:56.690473Z", "level": "info", "severity": "INFO", "event": "Original file stored"}
OURS {"source": "upload://aldermoor_procurement_policy.pdf", "chunks": 1, "timestamp": "2026-10-10T19:28:56.878942Z", "level": "info", "severity": "INFO", "event": "Embedding chunks and adding to vector store"}
LIBRARY HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-embedding-001:batchEmbedContents "HTTP/1.1 200 OK"
OURS {"filename": "aldermoor_procurement_policy.pdf", "pages": 1, "chunks": 1, "timestamp": "2026-10-10T19:28:59.593495Z", "level": "info", "severity": "INFO", "event": "Successfully ingested chunks"}
UVICORN INFO: 127.0.0.1:57707 - "POST /api/rag/upload HTTP/1.1" 200 OK
OURS {"query": "What payment terms do servo motor suppliers get?", "retriever_type": "similarity",
```

READ THE LINE

Our lines (JSON) [console yes] [log file yes]

event	Meridian AI starting	The message: what happened. In the line itself it is always the last field.
mode	local	local or cloud.
llm	gemini-3.8-flash	The chat model in use.
timestamp	2026-10-10T19:28:44.172480Z	When it happened, in UTC (the Z at the end).
level	info	info, warning or error, in lower case.
severity	INFO	The same level in capitals. Cloud Logging reads this field to colour and filter.

Our own line, written when the server starts (the lifespan function in main.py). It records the mode and the model.

Real lines from the runs of the previous chapter (10 October 2026). Folders are shortened to /Users/you/meridian-ai and long values to ... Click a line; use the filters to look at one kind.

// 26 LOGGING

Where each kind goes

KIND OF LINE	WRITTEN BY	FORMAT	CONSOLE	LOG FILE
Our lines	Our code: log.info(...)	One JSON object per line	yes	yes
Uvicorn lines	Uvicorn: start-up messages and one access line per request	Plain text starting INFO:	yes	no
Library lines	httpx and the Gemini client	Plain text	yes	yes

Checked, not assumed

After a run, grep -c "GET /api/health" logs/*.log prints 0, although the console showed that line.

Why the difference

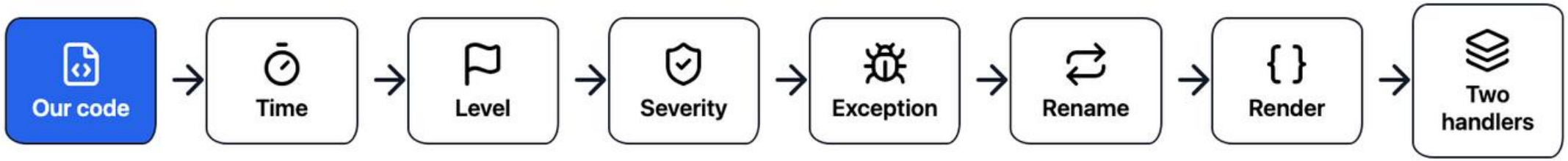
Our logger adds a console handler and a file handler to Python's root logger. Libraries pass their lines up to it. Uvicorn sets up its own loggers, which do not.

What to do

To keep access lines, read the console, or send it to a file when you start the server. We do not change Uvicorn's setup in this course.

// 26 LOGGING

How one line of ours is made



1. Our code calls the logger

A message and named values. Not a sentence with numbers inside: separate fields.

```
log.info("Meridian AI starting", mode=settings.app_mode, llm=settings.llm_model_name)
```

Back | Next step | Play

Step 1 of 8

// 26 LOGGING

Which event gets which level

EVENT (OUR LINES)	LEVEL	WRITTEN IN	THE CALLER SEES
Meridian AI starting; Frontend dist not found	info; warning	api/main.py	nothing; the API works
Original file stored; Successfully ingested chunks	info	rag/, endpoints.py	200
RAG query received; RAG answer generated	info	rag/retrieval.py	200
Unknown retriever_type, using similarity	warning	rag/retrieval.py	200, with a fallback
Starting audit phase; Tool call finished	info	agents.py, middleware.py	nothing
Configuration error; Ingestion error	error	api/main.py handlers	503; 422
Audit failed; RAG query failed	error	endpoints.py, with a traceback	500



the log holds full questions, answers and memos: treat log files as confidential

Read it: the file, the gap, the errors

project folder Copy

```
$ ls logs
10_11_2026_01_12_20.log
$ tail -n 3 "logs/$(ls -t logs | head -n 1)"
AFC is enabled with max remote calls: 10.
HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-
flash:generateContent "HTTP/1.1 200 OK"
{"memo": "...", "timestamp": "...", "level": "info", "severity": "INFO", "event": "CFO
Synthesis complete"}
$ grep -c "GET /api/health" logs/*.log
0
$ grep -h '"severity": "ERROR"' logs/*.log
{"error": "Could not read PDF 'upload://fake.pdf': Stream has ended unexpectedly", "timestamp":
"...", "level": "error", "severity": "ERROR", "event": "Ingestion error"}
```

> The file name is your local date and time (month, day, year, hour, minute, second); the times inside the lines are UTC.

ls logs

One file per server start. The name is your local time; the times inside are UTC.

Count access lines

0: the console showed them, the file never received them.

Find the errors

One line: the corrupt PDF of the previous chapter.

Read it: what the audit tools did

project folder Copy

```
$ grep -h "Tool call finished" logs/*.log | cut -c1-120
{"tool": "check_sanctions_list", "duration_ms": 3369, "preview": "CLEARED: Vendor not found on
known globa ...
{"tool": "get_vendor_credit_score", "duration_ms": 4730, ...
{"tool": "validate_fx_hedge", "duration_ms": 93, ...
{"tool": "calculate_cross_border_tax", "duration_ms": 5182, ...
{"tool": "categorize_expense", "duration_ms": 9046, ...
```

Five tool calls of one audit run through the API.

Five tool calls

Written by our middleware from Module 5.

Durations

The FX check takes under 0.1 s: it calls a real API. The others ask Gemini: usually 3 to 9 s, once 41 s.

cut

Only shortens each line to fit the screen.

// SUMMARY

Logging in five lines

1. **Our lines are JSON, built from fields.**
   Time, level, severity, message and named values.

2. **Two handlers: console and file.**
   Our lines and library lines reach both.

3. **Uvicorn's access lines reach the console only.**
   Checked with grep: 0 in the file.

4. **Errors match status codes.**
   503 missing setting, 422 bad file, 500 bug.

5. **The log is confidential.**
   It holds full questions, answers and memos.

// THE SERVICE AND THE SCREEN, PART FIVE

Tests that need no key

Seventeen tests that run in about five seconds, with no Gemini key, no internet and no Google Cloud



Why tests must not call Google

Model calls are slow

From a few seconds to a minute each.

They cost money

Every run would bill the key.

They differ every time

A test needs the same answer on every run.

So we fake the outside

Fake settings, a fake model, fake embeddings. Our own code runs for real.

// 27 TESTS

Put the tests in place

project folder

Copy

```
$ cp ../meridian-reference/backend/tests/conftest.py backend/tests/conftest.py
$ cp ../meridian-reference/backend/tests/test_api.py backend/tests/test_api.py
$ cp ../meridian-reference/backend/tests/test_modes_and_middleware.py
backend/tests/test_modes_and_middleware.py
$ wc -l backend/tests/*.py
  19 backend/tests/conftest.py
  80 backend/tests/test_api.py
 140 backend/tests/test_modes_and_middleware.py
 239 total
```

> Windows prints 19, 80 and 140 with the names (any __init__.py you made earlier shows as 0).

Three files
19, 80 and 140 lines.

backend/tests/
Already created in Module 2, with empty files.

Or type them
Follow the slides.

// 27 TESTS

conftest.py: fake settings first

backend/tests/conftest.py (all 19 lines)

```python
"""Test setup: fake config so the tests run without Google Cloud, a Gemini key or the
internet."""
import os
import sys
import tempfile

# Make `import api`, `import rag`, ... work when pytest is run from the project root
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))  1

# Environment variables win over .env, so a developer's real .env never leaks into tests
os.environ["APP_MODE"] = "local"  2
os.environ["LOCAL_DATA_DIR"] = tempfile.mkdtemp(prefix="meridian-test-")  3
os.environ["GOOGLE_API_KEY"] = "test-key"  4
os.environ["GCP_PROJECT_ID"] = "test-project"
os.environ["GCP_REGION"] = "us-central1"
# Empty on purpose: nothing may try to reach Cloud Storage
os.environ["GCS_BUCKET_NAME"] = ""  5
os.environ["GCS_PREFIX"] = "uploads/"
os.environ["VECTOR_SEARCH_INDEX_ID"] = ""
os.environ["VECTOR_SEARCH_INDEX_ENDPOINT_ID"] = ""
```

1. **Imports work**
   The backend folder is added to the path, so import api works from the project folder.

2. **Environment wins**
   Variables beat .env, so a real .env can never leak into a test.

3. **A temp folder**
   Local data goes to a fresh folder, not your real index.

4. **A fake key**
   The code is satisfied; Google would refuse it, so nothing may call Google.

5. **Cloud left empty**
   Nothing can reach Cloud Storage or Vertex AI.

test_api.py: routes with stand-ins

backend/tests/test_api.py (lines 8 to 46, shortened)

```python
client = TestClient(app)  1
# ...
def test_ask_returns_answer(monkeypatch):
    monkeypatch.setattr(endpoints, "ask_question", lambda query, retriever_type: f"answer to:
{query}")  2
    res = client.post("/api/rag/ask", json={"query": "hello", "retriever_type": "similarity"})
    assert res.status_code == 200
    assert res.json() == {"answer": "answer to: hello"}

# ...
def test_audit_returns_all_four_sections(monkeypatch):
    class FakeSupervisor:  3
        def run_audit(self, text):
            return {"risk_result": "r", "tax_result": "t", "control_result": "c", "cfo_memo":
"m"}

    monkeypatch.setattr(endpoints, "get_supervisor", lambda: FakeSupervisor())
    res = client.post("/api/agent/audit", json={"request_text": "buy 200 PLCs"})
    assert res.status_code == 200
    assert set(res.json()) == {"risk_result", "tax_result", "control_result", "cfo_memo"}
# ...
def test_upload_skips_non_pdf():
    res = client.post("/api/rag/upload", files={"files": ("notes.txt", b"hi", "text/plain")})
 4
    assert res.json()["results"][0]["status"] == "skipped"
```

1. **A client in memory**
   TestClient calls the app directly: no server, no port.

2. **A stand-in function**
   monkeypatch replaces ask_question for this test only. The route is tested, not Gemini.

3. **A fake supervisor**
   It returns four fixed strings, which proves the reply has the four fields.

4. **The real route**
   A .txt upload is skipped. The real code runs and never reaches Google.

// 27 TESTS

Seventeen tests, what each proves

test_api.py (9 tests) | test_modes_and_middleware.py (8 tests)

SHOW ONLY API | Settings | Ingestion | Agents | Logging

9 shown

test_health
The health route returns exactly status ok and the mode.

FAKED Nothing

API

test_status_reports_local_mode_without_calling_gcp
Status reports local mode, Chroma and the embedding model; no Google call.

FAKED Nothing

API

test_unknown_api_route_is_404
A path under /api that matches no route returns 404.

FAKED Nothing

API

test_ask_returns_answer
The ask route returns the answer for a valid body.

FAKED ask_question, replaced by a stand-in

API

test_audit_returns_all_four_sections
The audit route returns the four fields of AuditResponse.

FAKED The supervisor: four fixed strings

API

test_upload_skips_non_pdf
A .txt upload is skipped with a status, not an error.

FAKED Nothing: the real route runs

API

test_chunking_splits_long_text_and_keeps_metadata
A long text is cut into chunks of at most 1000 characters; each keeps its source.

FAKED Nothing

Ingestion

test_sample_pdfs_load_and_chunk
The four sample PDFs can be read and cut into chunks.

FAKED Nothing

Ingestion

test_log_lines_carry_a_severity_field_for_cloud_logging
add_severity copies the level into severity, and adds nothing if there is no level.

FAKED Nothing

Logging

// 27 TESTS

Four tests worth reading

backend/tests/test_modes_and_middleware.py (lines 58 to 133, shortened)

```
monkeypatch.setattr(settings, "GOOGLE_API_KEY", "")   1
llm.get_llm.cache_clear()
try:
    res = client.post("/api/rag/ask", json={"query": "hi"})
    assert res.status_code == 503   2
    assert "GOOGLE_API_KEY" in res.json()["detail"]
# ...
monkeypatch.setattr(vector_store, "get_embeddings", lambda: DeterministicFakeEmbedding(size=768))
 3
# ...
before = offline_local_store._collection.count()
ingest_pdf(pdf, "local://sample.pdf")
assert offline_local_store._collection.count() == before   4
# ...
model = FakeToolModel(messages=iter([   5
# ...
assert calls == ["Takumi"]
assert result["messages"][-1].text == "SANCTIONS CHECK: CLEARED"
assert "buyer@aldermoor.example" not in result["messages"][0].text
```

1. **A missing key**
   Blank the key for one test, call the route.

2. **503, not 500**
   The reply is a clear refusal, not a crash.

3. **Fake embeddings**
   The same text always gives the same numbers: a real index with no Google.

4. **No duplicates**
   Ingest the same PDF twice; the count must not change.

5. **A scripted model**
   It asks for the tool, then answers: the real middleware and the real tool run, and the e-mail never reaches the model.

// 27 TESTS

Run it: 17 passed

project folder

Copy

```
$ uv run pytest backend/tests
collected 17 items

backend/tests/test_api.py .........                                  [ 52%]
backend/tests/test_modes_and_middleware.py ........                  [100%]

============================== 17 passed in 4.12s ==============================
$ uv run pytest backend/tests -k missing_api_key -v
backend/tests/test_modes_and_middleware.py::test_missing_api_key_gives_503_not_500 PASSED
[100%]
====================== 1 passed, 16 deselected in 1.94s =======================
```

> Your screen shows other plugin lines in the header, and the time differs.

A dot is a pass

Nine and eight: one dot per test.

About five seconds

Ours: 4.1 s. No key, no internet, no Google call.

-k and -v

Run one test by part of its name, and print its name.

What these tests cannot tell you



A fake model scripts the plumbing

It proves that the code calls the model and uses its answer. It cannot say whether a real answer is good.



Retrieval quality is not tested

Whether the right chunks come back was judged by eye in the Module 4 lab.



The screen has one placeholder test

The frontend ships with a Vitest test that checks true is true. Nothing tests the tabs.



Run the real thing once

Passing tests do not replace one real audit (Module 5 lab and the next chapter).

// SUMMARY

Tests in five lines

1. **Real model calls are slow, paid and different each time.**
   Tests must not make them.

2. **conftest.py builds a fake world.**
   A real .env cannot leak in.

3. **Stand-ins replace only the outside calls.**
   Our own code runs for real.

4. **Seventeen tests pass in about five seconds.**
   No key, no internet.

5. **They check the plumbing, not the answers.**
   Run one real audit as well.

The screen: a React frontend

People need buttons, not curl. We copy a ready-made frontend, read the one file that talks to the server, and run it



// 28 THE SCREEN

The technology behind the screen

Build and run | Look and feel | Data and routing | Tests and checks

SHOW ONLY Core | Styling | Tooling | Installed, not used

4 shown

React

Used: version 18

A library that builds the screen from small components, each a function that returns what to show.

GOOD FOR Interactive pages.

Core

TypeScript

Used in every file

JavaScript with types, checked before the code runs. The same idea as type hints in Python.

GOOD FOR Catching mistakes early.

Core

Vite

Used: version 5, port 3000

The development server and the build tool. It reloads the page when you save, and builds the small final files.

GOOD FOR npm run dev and npm run build.

Tooling

Node.js and npm

Needed on your machine

Node runs JavaScript outside a browser; npm downloads the packages listed in package.json.

GOOD FOR Installing and running the tools.

Tooling

// 28 THE SCREEN

Copy the ready-made frontend

project folder

Copy

```
$ rsync -a --exclude node_modules --exclude dist ../meridian-reference/frontend/ frontend/
$ ls frontend
bun.lock  components.json  eslint.config.js  index.html  package-lock.json  package.json
playwright-fixture.ts  playwright.config.ts  postcss.config.js  src  tailwind.config.ts
tsconfig.app.json  tsconfig.json  tsconfig.node.json  vite.config.ts  vitest.config.ts
```

> The reference copy may hold a node_modules folder from its own build (about 340 MB): the exclusions leave it behind.

84 files

Of which 49 are generated components.

No packages yet

node_modules and dist are left behind: npm will recreate them.

Not hand-written

We read and run it; we do not write it in this course.

// 28 THE SCREEN

What is inside frontend/

frontend/

```
frontend/
├── index.html  # the one page; loads src/main.tsx
├── package.json  # packages and the npm scripts
├── vite.config.ts  # dev server on port 3000
└── src/
    ├── main.tsx  # starts React
    ├── App.tsx  # providers and the routes
    ├── pages/
    │   └── Index.tsx  # the sidebar and the four tabs
    ├── components/
    │   ├── DocumentUploadTab.tsx  # Index Documents (370 lines)
    │   ├── RagQATab.tsx  # RAG Q&A (186)
    │   ├── AuditTab.tsx  # Procurement Audit (382)
    │   ├── SystemStatusTab.tsx  # System Status (136)
    │   └── ui/  # 49 generated shadcn/ui files
    └── lib/
        └── api.ts  # the ONLY file that talks to the server
```

Four tabs, four files

One component per tab. About 1,100 lines in all.

ui/ is generated

49 files that shadcn/ui copied in. Nobody reads or edits them.

api.ts is the seam

Every call to the server goes through this one file.

A template

vite.config.ts loads a plugin called lovable-tagger: the project looks generated by a design tool, which we do not use.

// 28 THE SCREEN

api.ts: the only file that talks to the server

frontend/src/lib/api.ts (lines 5 to 52, shortened)

```ts
const BASE_URL = import.meta.env.VITE_API_URL || (import.meta.env.DEV ? "http://localhost:8080" :
""); 1
// ...
async function request<T = unknown>(path: string, options: RequestInit = {}): Promise<T> {
  const res = await fetch(`${BASE_URL}${path}`, options); 2
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.detail || `HTTP ${res.status}`); 3
  }
  return res.json();
}
// ...
export async function askQuestion(query: string, retrieverType: string = "similarity") { 4
  return request<{ answer: string }>("/api/rag/ask", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query, retriever_type: retrieverType }),
  });
}
// ...
export async function runAudit(requestText: string) { 5
  return request<{
```

1 Where the server is
In development: localhost:8080. In the built version: the same address the page came from (an empty string).

2 One helper for every call
It sends the request and reads the reply.

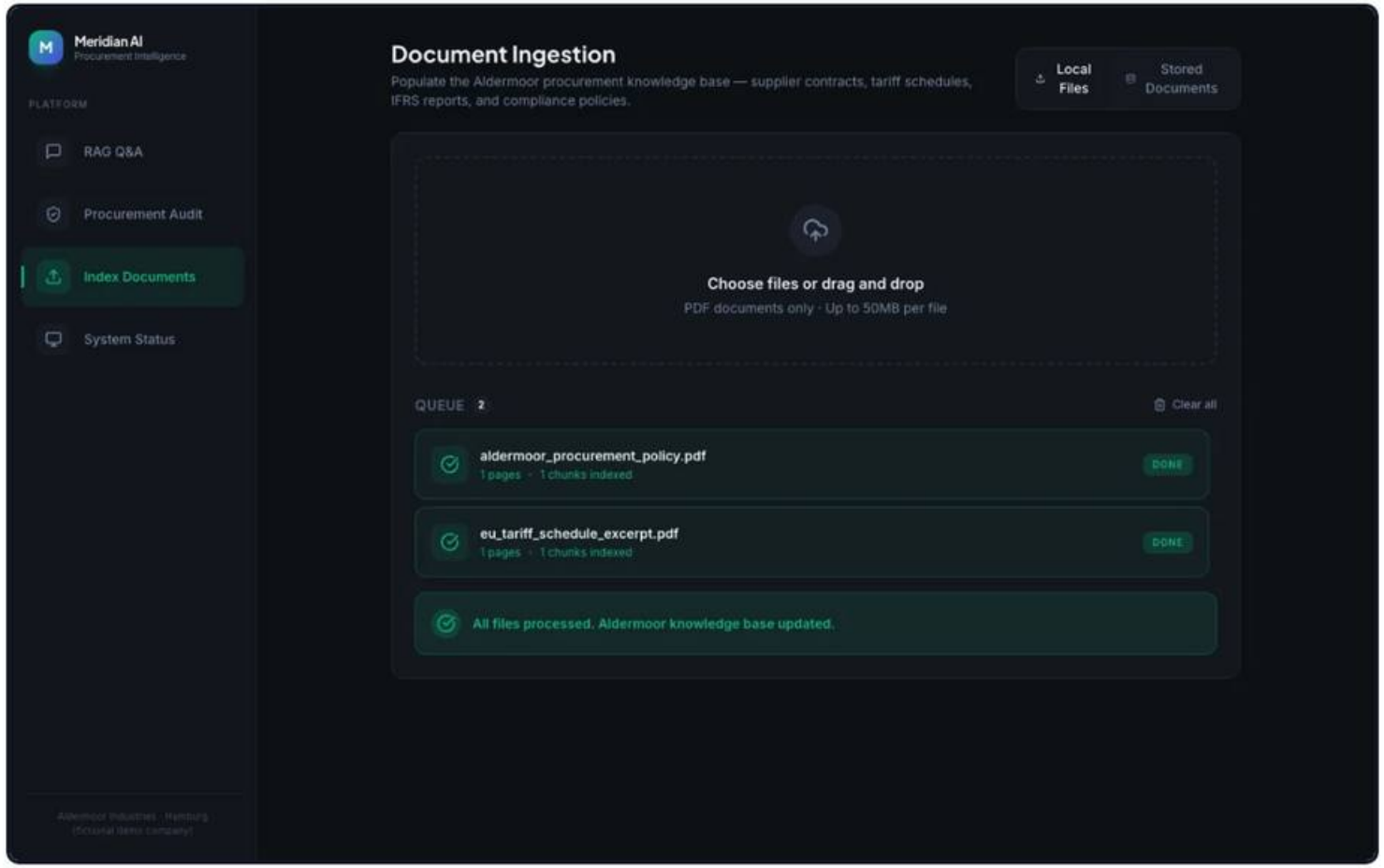
3 Errors keep their message
A failed reply becomes an Error with the server's detail text: that is how the 503 sentence reaches the screen.

4 One function per route
askQuestion sends the JSON that QueryRequest expects.

5 Same pattern
runAudit sends request_text, as AuditRequest expects.

// 28 THE SCREEN

Tab 1: Index Documents



The tab after two sample PDFs were uploaded (it opens first)

Staged, then sent

Files are queued and uploaded one at a time to /api/rag/upload.

A result per file

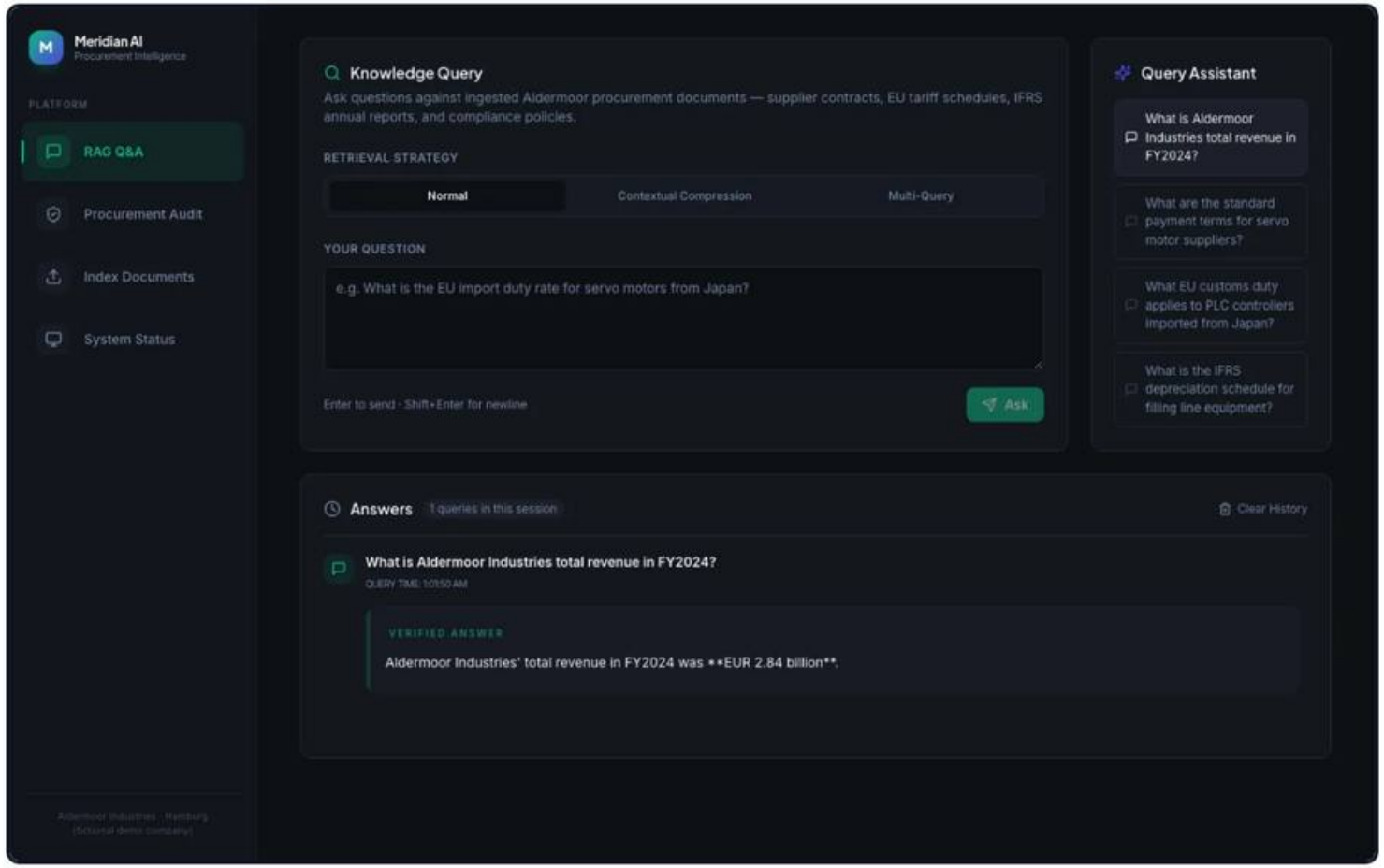
The page and chunk counts come from the reply.

Two small gaps

It says up to 50 MB per file, but nothing enforces that, and it prints 1 pages.

// 28 THE SCREEN

Tab 2: RAG Q&A



One question asked on the Normal strategy: 3.8 s in our run

Three strategies

Normal, Contextual Compression and Multi-Query: the retriever_type of Module 4.

The answer card

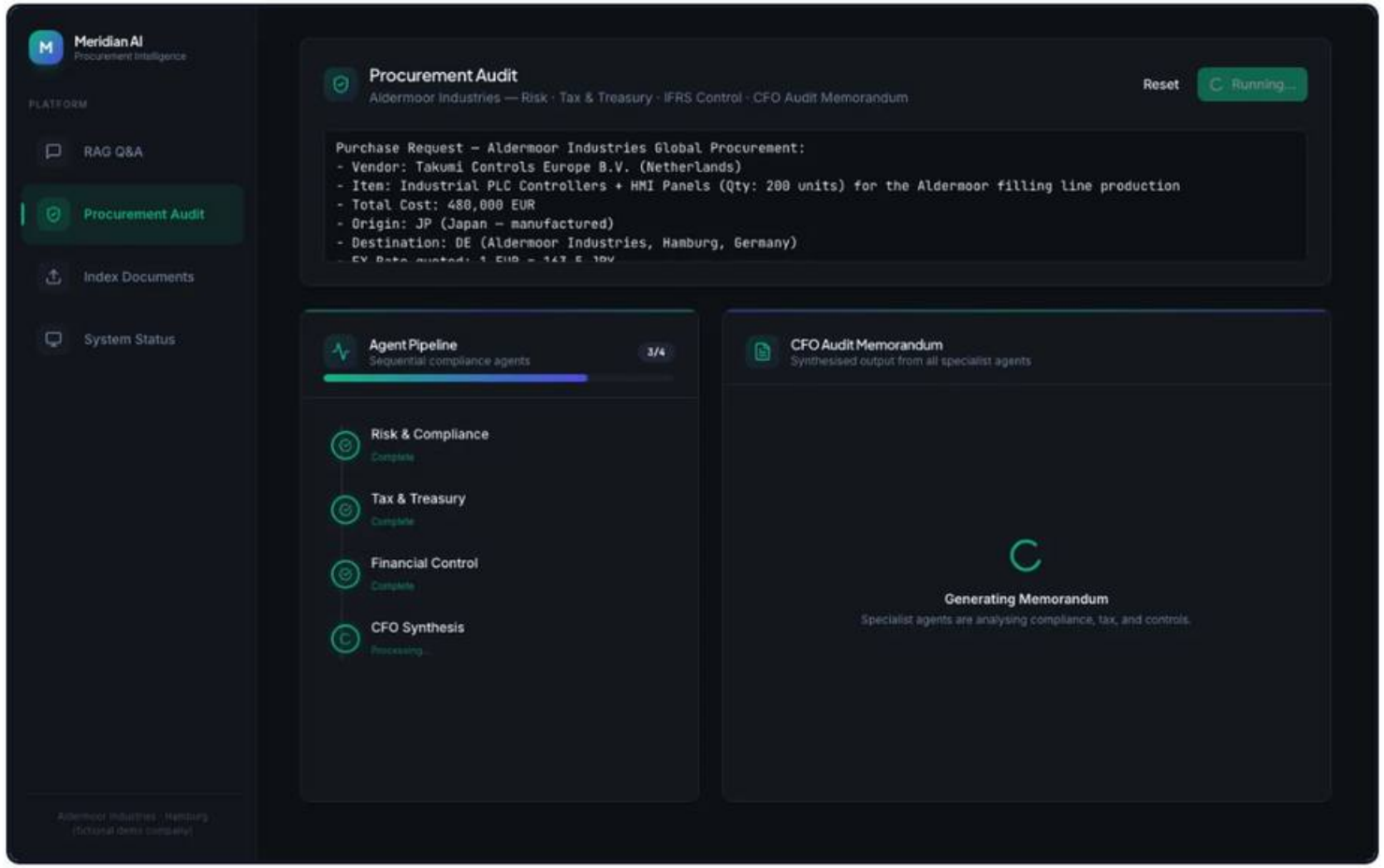
It is labelled Verified Answer, but nothing verifies it.

Raw marks

The model's ** marks show as asterisks: the page does not render Markdown.

// 28 THE SCREEN

Tab 3: Procurement Audit



12 seconds into an audit: three steps say Complete

Execute Audit

Sends the request text to /api/agent/audit and waits for the whole reply.

A minute

The audit takes 60 to 90 seconds; the screen shows the result only at the end.

Timers, not progress

The steps change at 3, 6 and 9 seconds, whatever the server is doing.

// 28 THE SCREEN

Why it says Complete too early

frontend/src/components/AuditTab.tsx (lines 62 to 81, shortened)

```
const timer1 = setTimeout(() => {  1
  setPhases((p) => p.map((ph, i) => (i === 0 ? { ...ph, status: "done" } : i === 1 ? { ...ph,
status: "running" } : ph)));
}, 3000);  1
// the same again at 6000 and 9000 ms
try {
  const data = await runAudit(request);  2
// ...
setMemo(data.cfo_memo);
setExpandedPhase("CFO Synthesis");  3
```

1 **A timer**
After 3 seconds step 1 is marked done, and step 2 starts. The same at 6 and 9 seconds.

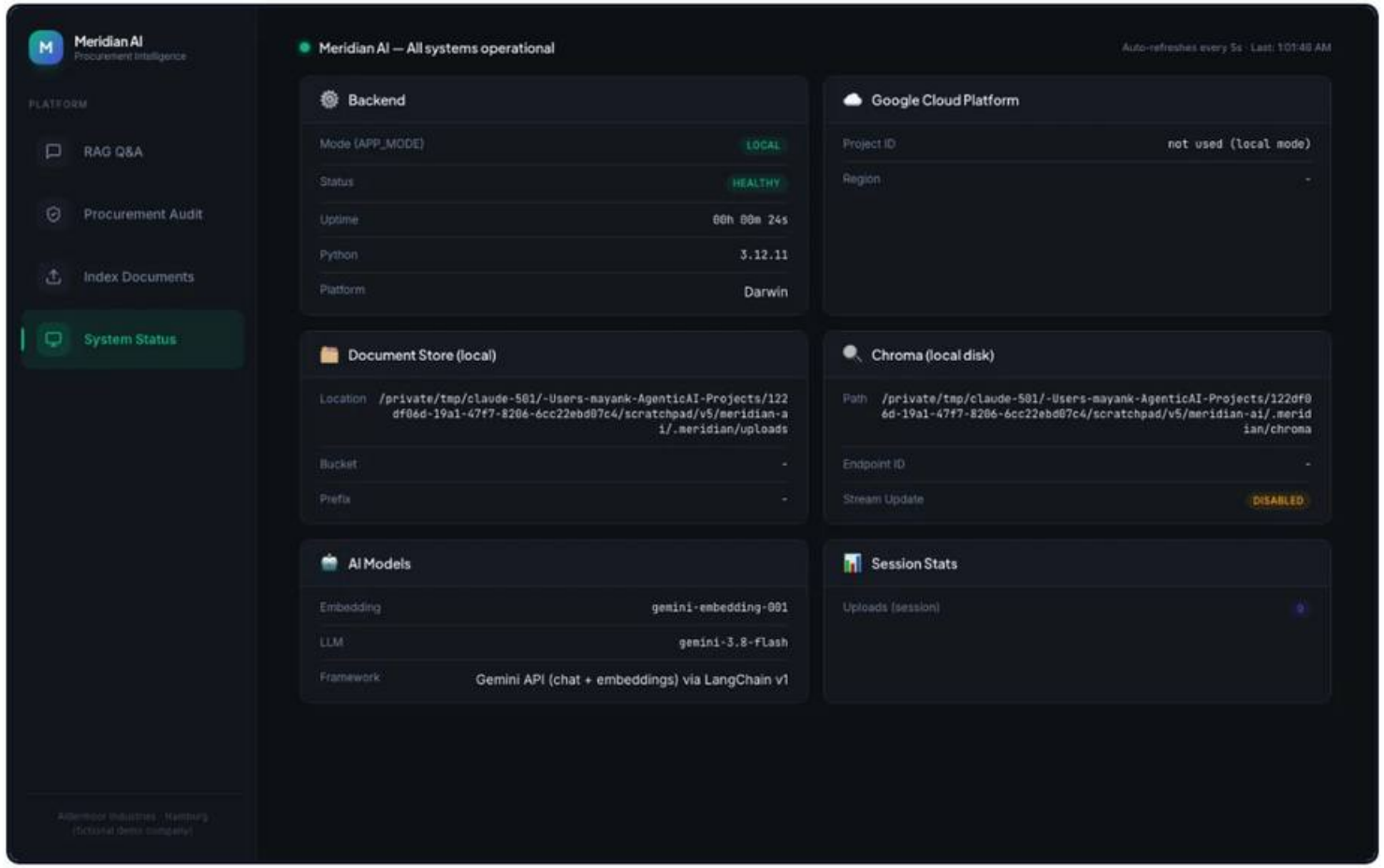
2 **The only real signal**
runAudit returns when the server has finished, after about a minute.

3 **Then the truth**
All four steps and the memo are filled in at once.

4 **Why it matters**
A progress display that is a guess misleads people. The server could send real progress; this one does not.

// 28 THE SCREEN

Tab 4: System Status



The tab as it looked on the scratch machine; yours shows your own folders

Polls every 5 seconds
setInterval calls /api/status again and again.

No key needed
The route asks nobody, so this tab works even with no key.

Your folders
The storage paths are your machine's. Look before you share a screenshot.

// 28 THE SCREEN

Run it: install and start

frontend/ (in the project folder) Copy

```
# run from the project folder, then go into frontend/
$ cd frontend
$ npm install
added 499 packages, and audited 500 packages in 6s
88 packages are looking for funding
33 vulnerabilities (1 low, 11 moderate, 18 high, 3 critical)

$ npm run dev
  VITE v5.4.21  ready in 203 ms
  ➜  Local:   http://localhost:3000/
```

> Dependencies change over time, so your numbers differ.

npm install

Downloads about 340 MB into node_modules. The first time it can take a minute.

33 vulnerabilities

The warning is normal for this template. Run npm audit before deploying anything.

npm run dev

Open http://localhost:3000. The server of the previous chapter must be running for the buttons to work.

// SUMMARY

The screen in five lines

1. **React builds it, Vite runs it, Tailwind styles it.**
   TypeScript checks it; shadcn/ui supplies the parts.

2. **api.ts is the only file that talks to the server.**
   One function per route, with our field names.

3. **Four tabs: upload, ask, audit, status.**
   Each is one component.

4. **Three honest gaps.**
   A Verified label that verifies nothing, raw asterisks, and progress that is a timer.

5. **Port 3000 needs the server on port 8080.**
   Or build it, and let the server serve it: the last chapter.

// THE END OF PART 1

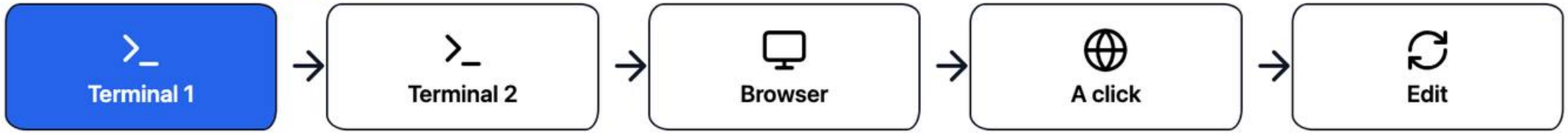
Run it all, locally

Two ways to run the whole system on your machine, a lab to prove it works, and a look back at what you built



Two ways to run it

Mode 1: development, two terminals | Mode 2: one server, built



1. Start the API

Uvicorn serves the routes on port 8080, and restarts when you save a Python file.

```
uv run uvicorn api.main:app --app-dir backend --reload --port 8080
```

Back | Next step | Play

Mode 1: development, two terminals: step 1 of 5

// 29 RUN IT ALL

Before you start

CHECK	HOW YOU KNOW
The key is in .env	Ask a question in the screen: a 503 sentence means it is missing
The index exists	Run the four sample PDFs through the Index Documents tab, or the upload of the earlier chapter
Ports 8080 and 3000 are free	Uvicorn says Address already in use if not
uv and Node are installed	uv --version and node --version both print a number
You are in the project folder	ls shows backend, frontend and pyproject.toml

never show .env or the key

// 29 RUN IT ALL

Mode 1: two terminals

project folder Copy

```
# terminal 1: the API (project folder)
$ uv run uvicorn api.main:app --app-dir backend --reload --port 8080
INFO:     Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
# terminal 2: the screen
$ cd frontend
$ npm run dev
  ➜  Local:   http://localhost:3000/
# open http://localhost:3000
```

> The screen on port 3000 calls the API on port 8080 (src/lib/api.ts picks that address when it runs in development mode).

Terminal 1
The API. Leave it running.

Terminal 2
The screen. Leave it running.

Open port 3000
http://localhost:3000, with the API on port 8080.

// 29 RUN IT ALL

The lab: use the whole app

	WHERE	WHAT TO DO	WHAT YOU SHOULD SEE
1	Index Documents	Choose the four PDFs from sample_docs, then start the ingestion	Four rows say DONE: 1 page, 1 chunk indexed each
2	RAG Q&A	Click the second example question	A Net-45 answer for suppliers with a credit score of 75 or higher (our run: 3.8 s)
3	RAG Q&A	Ask the same question on each of the three strategies	Similar answers; compare how long each takes
4	Procurement Audit	Press Execute Audit with the sample request	About 70 s. The steps say Complete early (timers). Our runs ended with the payment held
5	System Status	Open the tab	Mode local; the two models; your uploads this session
6	Break it	Stop the API (Ctrl+C), then ask a question	Error Encountered: Failed to fetch; the Status tab says Backend Unreachable

// 29 RUN IT ALL

Mode 2: one server

project folder

Copy

```
$ cd frontend
$ npm run build
✓ built in 2.43s
$ cd ..
$ uv run uvicorn api.main:app --app-dir backend --port 8080
INFO:     Started server process [4793]
INFO:     Waiting for application startup.
{"mode": "local", "llm": "gemini-3.8-flash", ..., "event": "Meridian AI starting"}
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8080 (Press CTRL+C to quit)
# open http://localhost:8080
```

> Now the warning about the missing frontend is gone: main.py found frontend/dist and serves it.

Build once
1,681 files become three. The build takes a few seconds.

No --reload
One process, one port, 8080.

The warning is gone
main.py found frontend/dist.

// 29 RUN IT ALL

How the single server answers

project folder Copy

```
$ curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://localhost:8080/
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/some/page
$ curl http://localhost:8080/api/nothing
200 text/html; charset=utf-8
200
{"detail":"Not found"}
```

> Any page that is not a file returns the React app (React decides what to show).

Any page: the app

React decides what to show for a path such as /some/page.

api/ stays JSON

Not found, with a small f, comes from our catch-all.

No escaping

A path with ../ returns the app, never a file.

// 29 RUN IT ALL

Test and commit

project folder

Copy

```
$ uv run pytest backend/tests
============================== 17 passed in 4.12s ==============================
$ cd frontend
$ npm test
      Tests  1 passed (1)
$ cd ..
$ git add -A
$ git commit -m "The service and the screen: API, tests, frontend"
[main 4d74292] The service and the screen: API, tests, frontend
 90 files changed, 16537 insertions(+)
$ git status --short
# prints nothing: everything is committed
```

> Our commit: 90 files, 16,537 insertions; your commit code differs.

Two test suites

17 backend tests, and one placeholder frontend test.

Everything in

API, tests, frontend and the try scripts. node_modules, dist, .env, .meridian and logs stay out of Git.

90 files

Most of the lines are package-lock.json and bun.lock. Your commit code differs.

What Part 1 built

```
meridian-ai/
meridian-ai/
├── backend/
│   ├── config/  # settings from .env
│   ├── logger/  # JSON logs, console and file
│   ├── exceptions/  # our own errors
│   ├── rag/  # document question and answer
│   ├── agent/  # tools, prompts, middleware, the audit team
│   ├── api/  # the server: shapes, routes, main
│   └── tests/  # 17 tests
├── frontend/  # the React screen
├── sample_docs/  # the four sample PDFs
├── .env  # your key: never in Git
├── .meridian/  # your index and uploads: not in Git
└── logs/  # one file per server start: not in Git
```

Ask your documents

Upload PDFs and ask questions; answers come from the pages.

Audit a purchase

Three specialists and a CFO, with guard-rails and a log.

Use it in a browser

One server, or two terminals.

Change it safely

Seventeen tests that need no key.

// 29 RUN IT ALL

What it does not do

LIMIT	WHAT IT MEANS
Four of the five audit tools use the model's memory	Only the exchange-rate check uses live data. A real sanctions list or credit agency would go behind the same tools.
The agents never read your documents	Document Q&A and the audit are separate features.
The supervisor is plain Python; the rules are in a prompt	Three agents in a fixed order, and a decision rule you can change by editing text.
The screen overclaims	A Verified label that verifies nothing, raw asterisks, progress that is a timer.
No login; CORS allows any site; history is in memory	Fine for a demonstration. Not for a shared server. The 50 MB limit on the screen is not enforced.
Only PDFs with text work	A scanned page is skipped with a reason.
Wording changes on every run	The decision words and the keywords do not.

// SUMMARY

Part 1 in five lines

1 **A browser, a server, an agent team and an index.**
All on your machine, with one key.

2 **The server is thin.**
Shapes, routes, handlers: the logic lives in rag/ and agent/.

3 **Every failure has a clear reply and a log line.**
503, 422, 500, and an error in the log.

4 **Seventeen tests prove the plumbing.**
One real audit proves the rest.

5 **Next: Part 2.**
Put this exact system in a container, then in the cloud.